

Projet Informatique Électronique 2017

EI2I3

Explications

Cette partie présente l'arborescence des différentes parties du projets. Ces parties étant étroitement liées, je vous recommande de respecter cette organisation ainsi que les noms de fichiers quand vous rendrez vos projets (dernière semaine de juin).

Voici les différents dossiers et fichiers que vous trouverez :

```
remi@remi-VirtualBox:~/Desktop/Dropbox/Work/UPMC/ProjetInfoElec/ProjetInfoElec-EI2I3$ tree
.
├── asm
│   ├── asm.c
│   └── asm.h
├── ast
│   ├── ast.c
│   └── ast.h
├── bison
│   ├── grafcet.y
│   ├── y.dot
│   ├── y.tab.c
│   ├── y.tab.h
│   └── y.tab.h.gch
├── build
├── graf
│   ├── basic2.grafcet
│   ├── basic.grafcet
│   ├── ex1.grafcet
│   ├── ex2.grafcet
│   └── safe.grafcet
├── grafInterpreter
│   └── graphInterpreter.c
├── lex
│   ├── grafcet.l
│   └── lex.yy.c
├── Makefile
├── ReadMe.pdf
├── vm
│   ├── evm.c
│   └── vm_codops.h
```

Jusqu'à maintenant, nous avons codé :

- **la machine virtuelle** : Située dans le dossier **vm**. Elle exécute le bytecode (.bin) généré par l'assembleur
- **un interpréteur de grafcet** : Situé dans le dossier **grafInterpreter**. Ce programme est à part de notre chaîne de compilation-assemblage. Il est une traduction en C de l'algorithme d'interprétation d'un grafcet.
- **l'assembleur** : Situé dans **asm**. Ce programme génère le bytecode à partir du code assembleur.

Afin de compléter notre chaîne, il ne nous manque plus que le compilateur (générant de l'assembleur à partir du grafcet), celui ci se répartie entre :

- l'analyseur syntaxique : Situé dans le dossier **bison**
- l'analyseur lexical : Situé dans le dossier **lex**
- Constructeur et interpréteur de l'arbre syntaxique : Situé dans le dossier **ast**
- les fichiers source grafcet : Situé dans le dossier **grafcet**

Deux scripts sont mis à votre disposition pour générer tout les fichiers nécessaires. Il s'agit de **built** et **Makefile**.

Pour construire tout votre projet, de la compilation jusqu'à l'exécution, en passant par l'assemblage, il suffira de lancer **./built** en console. Celui ci va lancer bison puis lex. Ces 2 étapes sont transparentes pour vous. Il va ensuite générer l'exécutable **grafcet.out** grâce à la commande **Make grafcet.out**. Il va ensuite exécuter ce dernier. Le résultat est le fichier **safe.asm** (si l'on choisi pour fichier grafcet **./graf/safe.grafcet**).

De même avec **asm.out** (génération puis exécution). Le résultat étant **safe.bin**

Travail demandé

Il vous est demandé ici de :

1. prendre en main les scripts (**build** et **makefile**) et de compléter les dossiers **vm**, **grafinterpreter** et **asm** avec ce que vous avez réalisé jusqu'à maintenant. A nouveau, il est préférable de respecter les noms des fichiers pour que les scripts fonctionnent.
2. compléter dans **./ast/ast.c**, les fonctions **makedual** et **generatecode**.
Makedual : Cette fonction va construire l'arbre syntaxique, en partant des structures définies dans **ast.h**. J'ai placé des balises « **ToDo** » en commentaires à chaque fois qu'il y une instruction à écrire.
N'hésitez pas à utiliser les fonctions d'affichages des champs des structures pour vérifier votre travail.
Generatecode : Elle exploite l'arbre réalisé précédemment afin de générer le code. Cette fonction ne contient que les premières instructions, à vous de la compléter.
Ces deux fonctions sont appelées par la fonction main de **./bison/y.tab.c**
3. Un rapport vous sera également demandé, documentant vos sources. Vous pouvez l'effectuer avec Doxygen (format html). Des balises doxygen sont présentes dans le fichiers **asm/asm.c** à titre d'exemple.